CALIBRATED ONLINE REPAIR MEMORY FOR REPEATED ROBOT DEPLOYMENT: A MUJOCO STRESS TEST

Anonymous authors
Paper under double-blind review

ABSTRACT

Robots deployed repeatedly in the same environment encounter model errors that are neither fully random nor fully captured by a nominal simulator: contact friction can be shifted, actuation can have a persistent gain or angle bias, and obstacles can be displaced. A tempting response is to store previous embodied corrections and reuse them as a repair memory. Earlier versions of this project tested that idea with a nearest-neighbor residual cache and failed. This paper rebuilds the idea into a falsifiable CPU-only MuJoCo benchmark and a stronger method, Calibrated Online Repair Memory (CORM), that treats repair memory as online residual system identification with uncertainty-aware trust and a selective safety shield. We freeze a hostile protocol with 25344 main raw rows, 5632 ablation raw rows, nine deployment shifts, strong MPC and memory baselines, paired bootstrap intervals, sign-flip tests, learning curves, and nonstationary stress. The frozen terminal decision is **KILL_ARCHIVE**. CORM reaches 16.6% aggregate success, 0.149 oracle regret, and 28.8% violation rate. We do not claim ICLR-main readiness: without hardware or public benchmark evidence, the correct interpretation is either a strong-revise simulation result or a rigorous kill/archive result, exactly as determined by the frozen gate.

1 INTRODUCTION

Model error is often treated as a nuisance to be averaged away, bounded by robust control, or absorbed into a larger learned policy. In repeated robot deployment, however, the same error can appear again and again. A puck slides farther than the nominal contact model predicts on a low-friction surface; a lightweight object rotates more than expected; a pusher has a small angle bias; a calibration error shifts an obstacle. A system that sees these errors once should not necessarily start from zero on the next task.

This observation motivates embodied repair memory: store the residual between predicted and observed outcomes, then use that residual to select future actions. The idea is compatible with the model-based planning tradition (Mayne et al., 2000; Williams et al., 2017; Chua et al., 2018), online system identification (Ljung, 1999; Anderson & Moore, 1979), and memory-based reasoning (Aamodt & Plaza, 1994). It is also dangerous. A residual that helped one contact may be irrelevant for another; a stale memory can survive after a hidden deployment change; a correction that improves final distance can increase obstacle violations. A submission-quality paper therefore cannot merely show a pretty learning curve. It must show where repair memory helps, where it fails, and whether a proposed calibration mechanism is actually better than strong simple baselines.

The starting point of this work was negative. The previous v4 version implemented a real MuJoCo sequential contact benchmark, but the proposed kNN repair memory did not beat nominal MPC, robust worst-case MPC, or simple global and last-repair memories. That result killed the original claim. We therefore do not present CORM as a cosmetic rename of the old method. We rebuild the mechanism around a narrower thesis:

> Repeated deployment streams can contain reusable residual structure, but repair memories should be reused only through calibrated online residual models that estimate uncertainty and reduce trust under stale or unsafe evidence.

The central question is not whether memory can be made to look good. The question is whether it survives a hostile review protocol: all methods share the same candidate action library, the same MuJoCo rollouts, the same hidden deployment streams, and the same seeds. The proposed method is compared with random selection, nominal MPC, robust worst-case MPC, last repair, global average repair, the old kNN repair memory, an online ridge residual model without CORM calibration, CORM ablations, and an oracle that is allowed to evaluate the true hidden deployment.

This paper makes four contributions.

A falsifiable repeated-deployment benchmark. We define a planar MuJoCo pushing task with persistent hidden shifts in mass, friction, actuation gain, angle bias, lateral bias, and obstacle placement. The final protocol has nine stress splits, including a nonstationary split that changes the hidden deployment halfway through the stream.

A calibrated residual-memory method. CORM stores compact tabular residual observations and fits an online ridge residual model. It predicts displacement and violation residuals, estimates uncertainty from residual error and leverage, blends nominal and repaired predictions with a trust coefficient, and activates a selective safety shield only under high predicted risk.

Theory for both the positive and negative cases. We give simple deterministic statements showing when residual memory can improve planning, when calibrated trust bounds damage, and why stale memory can be worse than robust or reset baselines under unobserved change points.

A frozen evidence package. The result package includes raw rows, ablations, learning curves, paired statistics, trust diagnostics, generated tables, validation scripts, and a terminal decision. The final decision is **KILL_ARCHIVE**; the paper is not presented as ICLR-main ready.

2 Problem Setup

2.1 Repeated deployment with hidden physical parameters

We consider a stream of manipulation tasks indexed by $t = 1, \ldots, T$. At each task, the robot observes a state x_t, target g_t, obstacle geometry o_t, and a finite candidate action set $\mathcal{A}_t$. The robot has a nominal simulator f_0 and cost c_0, but the real deployment is governed by a hidden parameter θ_t and dynamics f_{θ_t}. The hidden parameter may persist across a stream, as in ordinary repeated deployment, or change at an unobserved phase boundary, as in our nonstationary stress split.

In our benchmark, x_t is the puck position, g_t is a target point, o_t is a cylindrical obstacle, and $a \in \mathcal{A}_t$ is a parameterized push primitive. The nominal model uses fixed mass and friction. The true deployment changes mass, friction, actuation distance gain, angular bias, lateral bias, and obstacle displacement. Outcomes are generated by MuJoCo (Todorov et al., 2012).

For a candidate action a, the nominal simulator predicts a final puck position $\hat{y}_0(x_t, a)$ and violation indicator $\hat{v}_0(x_t, a)$. The hidden deployment produces $y_\theta(x_t, a)$ and $v_\theta(x_t, a)$. The residuals are

$$r_t(a) = y_\theta(x_t, a) - \hat{y}_0(x_t, a), \tag{1}$$

$$\rho_t(a) = v_\theta(x_t, a) - \hat{v}_0(x_t, a). \tag{2}$$

The robot observes these residuals only for the action it executes.

2.2 Decision objective

All methods choose one candidate action per task. The evaluation energy is

$$E(y, v, e; g) = \|y - g\|_2 + 0.30v + 0.03e, \tag{3}$$

where e is pusher path effort. Success requires final distance at most 0.075 and no safety violation. Regret is computed against a hidden-deployment oracle that evaluates all candidates under f_{θ_t} and chooses the minimum true energy action. This oracle is not a deployable policy; it is an upper bound.

2.3 Why the old kNN memory failed

The v4 kNN memory stored (ϕ_t, r_t, ρ_t) tuples and retrieved nearby residuals by Euclidean distance in hand-designed features ϕ_t. It failed for three reasons. First, residual reuse is a system-

identification problem, not pure nearest-neighbor recall. A lateral bias or gain error induces structured residuals across many actions; kNN retrieval can ignore that structure. Second, the old memory had no calibrated trust. It could use a correction with the same confidence on episode 3 as on episode 30. Third, it had no stale-memory mechanism. A memory that is useful before a hidden change point can become harmful after it.

3 CALIBRATED ONLINE REPAIR MEMORY

3.1 FEATURE REPRESENTATION

For each candidate, CORM constructs a compact feature vector $\phi(x_t, g_t, o_t, a, \hat{y}_0)$ containing the action angle relative to the target direction, push distance, lateral offset, target displacement, nominal final displacement, nominal target error, geometric endpoint error, line-obstacle clearance, obstacle distances, nominal violation, nominal clearance margin, and effort. The feature vector is small and stored as a dense NumPy array. No neural network or GPU is used.

3.2 ONLINE RESIDUAL REGRESSION

CORM maintains a bounded memory $\mathcal{D}_t = \{(\phi_i, z_i)\}_{i<t}$, where

$$z_i = [r_i^{(x)}, r_i^{(y)}, \rho_i]^\top. \tag{4}$$

At each update, it fits a ridge model

$$\hat{B}_t = (\Phi_t^\top \Phi_t + \lambda I)^{-1} \Phi_t^\top Z_t. \tag{5}$$

This is a deliberate CPU-light choice. The goal is not to win by capacity; it is to test whether a calibrated residual object can beat simpler memories under a fair action library. A deep memory model would be a different paper and would require substantially more evidence.

3.3 TRUST AND UNCERTAINTY

For a candidate with feature ϕ, CORM predicts $\hat{z} = \phi^\top \hat{B}_t$. It also computes an uncertainty proxy

$$u_t(\phi) = \mathrm{RMSE}_t + \beta \sqrt{\phi^\top (\Phi_t^\top \Phi_t + \lambda I)^{-1} \phi} + \frac{\gamma}{\sqrt{|\mathcal{D}_t|}}, \tag{6}$$

where RMSE_t is the residual error on stored observations. The trust coefficient is

$$\alpha_t(\phi) = \min\{\alpha_{\max}, (1 - \exp(-|\mathcal{D}_t|/\tau)) \exp(-\kappa u_t(\phi))(1 - s_t)\}, \tag{7}$$

where s_t is a shock variable increased by unexpectedly large residual prediction errors and decayed after stable observations. The repaired prediction is

$$\hat{y}_t^{\mathrm{rep}}(a) = \hat{y}_0(a) + \alpha_t(\phi) \hat{r}_t(a). \tag{8}$$

3.4 SELECTIVE SAFETY SHIELD

CORM predicts a violation risk by combining nominal violation, predicted violation residual, uncertainty, and nominal clearance. A development run exposed an important failure: continuous low-confidence risk penalties can harm action selection even when no safety intervention is justified. The frozen method therefore uses a selective shield. It applies an additional risk and robust-score penalty only when predicted risk exceeds a high threshold. This makes the safety mechanism auditable: shield activation is reported, and the no-safety ablation is run on the same tasks.

3.5 BASELINES AND ABLATIONS

The baseline suite is intentionally strong for a small contact benchmark. Nominal MPC chooses the lowest nominal energy action. Robust worst-case MPC evaluates each action under low-friction, nominal, and high-friction branches and minimizes the worst predicted energy. Last repair and global repair store simple residual summaries. The old kNN memory is retained as 'knn_repair_memory_v4'. Online ridge residual uses the same residual regression family but removes CORM's calibrated uncertainty and safety behavior. Ablations remove safety, remove uncertainty, remove context features, and restrict memory capacity.

4 THEORY: WHEN REPAIR MEMORY HELPS AND WHEN IT HURTS

The following statements are deliberately simple. They are not intended to prove that CORM is universally optimal. They clarify what any residual-memory method must satisfy to be worth submitting.

Definition 1 (Residual planning error)**.** *For a candidate a, define the nominal prediction error $b(a) = y_\theta(a) - \hat{y}_0(a)$ and the residual-estimation error $\epsilon_t(a) = b(a) - \hat{r}_t(a)$. A repaired planner with trust $\alpha \in [0, 1]$ predicts $\hat{y}_\alpha(a) = \hat{y}_0(a) + \alpha\hat{r}_t(a)$.*

Lemma 1 (Trust-bounded prediction error)**.** *For any action a and trust α,*

$$\|y_\theta(a) - \hat{y}_\alpha(a)\|_2 \leq (1 - \alpha)\|b(a)\|_2 + \alpha\|\epsilon_t(a)\|_2. \tag{9}$$

Proof. Substitute $y_\theta(a) = \hat{y}_0(a) + b(a)$ and $\hat{y}_\alpha(a) = \hat{y}_0(a) + \alpha\hat{r}_t(a)$. The error is $(1 - \alpha)b(a) + \alpha(b(a) - \hat{r}_t(a))$. The result follows from the triangle inequality. □

The lemma exposes the core tradeoff. Trust helps only when residual-estimation error is smaller than nominal bias. If $\|\epsilon_t(a)\|_2$ is large, a smaller α is safer. This is why CORM reports trust and uncertainty rather than treating memory as always beneficial.

Proposition 1 (Sufficient condition for residual improvement)**.** *Consider two candidates $a^\star$ and a_0, where $a^\star$ has lower true target distance than a_0 by margin $\Delta > 0$. If repaired prediction errors for both candidates are bounded by $\eta < \Delta/2$, then minimizing repaired target distance selects an action whose true target distance is no worse than $a^\star$ by at most 2η.*

Proof. Let $d(a) = \|y_\theta(a) - g\|_2$ and $\hat{d}(a) = \|\hat{y}_\alpha(a) - g\|_2$. If $|\hat{d}(a) - d(a)| \leq \eta$ for all compared actions, then $\hat{d}(a^\star) \leq d(a^\star) + \eta$. Any selected action $\hat{a}$ satisfies $\hat{d}(\hat{a}) \leq \hat{d}(a^\star)$. Therefore $d(\hat{a}) \leq \hat{d}(\hat{a}) + \eta \leq d(a^\star) + 2\eta$. □

This is the positive case: if repeated deployment makes residuals predictable enough, memory can move the planner toward the hidden-deployment oracle. The proposition also explains why a large action library is not enough. More candidates help only if repaired predictions rank them accurately.

Theorem 1 (Stale-memory harm under hidden change points)**.** *There exists a two-action repeated-deployment problem with a single unobserved change point such that a persistent residual memory with fixed positive trust has strictly higher cumulative regret after the change point than a reset or robust baseline.*

Proof. Let the pre-change residual for action a_1 be $+\delta$ and for a_2 be 0, and let target geometry make a_1 optimal after applying the $+\delta$ correction. After the hidden change point, reverse the residual sign for a_1 to $-\delta$ while leaving nominal predictions unchanged. A persistent memory with trust $\alpha > 0$ continues to add a positive correction and ranks a_1 too favorably for enough episodes before adapting. A reset baseline has no stale correction, and a robust baseline that hedges both residual signs does not commit to the stale direction. Choosing δ larger than the nominal action margin gives strictly positive extra regret for the persistent memory on each stale episode. □

The theorem is not a pathology; it is the nonstationary split in miniature. A credible repair-memory paper must include stale-memory stress tests.

Proposition 2 (Safety-risk decomposition)**.** *Let $\hat{v}_0(a)$ be nominal violation, $\hat{\rho}_t(a)$ be predicted violation residual, and $e_v(a) = \rho_t(a) - \hat{\rho}_t(a)$. Then the true violation residual satisfies*

$$v_\theta(a) \leq \hat{v}_0(a) + \alpha\hat{\rho}_t(a) + |e_v(a)| + (1 - \alpha)|\rho_t(a)|. \tag{10}$$

Thus safety can fail because the nominal model is unsafe, the residual model is wrong, trust is too high, or the action library contains no safe high-quality primitive.

Proof. Write $v_\theta = \hat{v}_0 + \rho_t = \hat{v}_0 + \alpha\hat{\rho}_t + (\rho_t - \alpha\hat{\rho}_t)$ and bound the remaining term by adding and subtracting $\alpha\rho_t$. □

This decomposition motivates reporting both violation rate and shield activation. A method that improves final distance by taking unsafe actions is not submission-ready.

Table 1: Aggregate frozen metrics across all nine main splits.

Method	Success	Regret	Violation	Trust	Shield
random_candidate	2.0	0.322	48.4	0.000	0.0
nominal_mpc	13.1	0.173	34.7	0.000	0.0
robust_worst_case_mpc	12.1	0.145	26.1	0.000	0.0
last_repair_memory	13.0	0.172	34.7	0.054	0.0
global_average_repair	16.1	0.163	34.6	0.403	0.0
knn_repair_memory_v4	17.2	0.155	32.7	0.408	0.0
online_ridge_residual	18.1	0.149	30.3	0.788	0.0
corm_repair_memory_v5	16.6	0.149	28.8	0.435	1.4
corm_no_safety	17.0	0.153	32.6	0.433	0.0
corm_no_uncertainty	16.6	0.140	22.1	0.788	4.5
oracle_hidden_deployment	50.9	0.000	0.2	0.000	0.0

5 Experimental Protocol

5.1 Task and action library

The MuJoCo scene contains a cylindrical puck, a spherical pusher, and a cylindrical obstacle. Each candidate action is a straight push primitive parameterized by angle, lateral offset, and push distance. The frozen action library uses seven target-relative angles, three lateral offsets, and three distance scales, giving 63 candidates per task. All methods choose from the same candidates.

For each candidate, the runner evaluates a nominal rollout, a hidden-deployment rollout, and robust branch rollouts. These rollouts are shared by all methods for that task, which prevents a method from receiving extra simulation budget. Only the oracle can use hidden-deployment energies for action selection.

5.2 Frozen splits

The final run uses nine main splits: nominal, low friction, high friction, heavy object, light object, actuation bias, obstacle shift, nonstationary deployment shift, and combined shift. Each split changes a different part of the hidden deployment. The combined shift samples broad changes across mass, friction, gain, angle bias, lateral bias, and obstacle displacement. The nonstationary split deliberately changes the hidden deployment halfway through the stream.

5.3 Evidence size and statistics

The frozen command uses 8 seeds, 32 episodes per seed/split/method, 9 main splits, and 11 main methods, producing 25344 main raw rows. Ablations are run on combined and nonstationary splits, producing 5632 ablation rows. We report mean success, final distance, violation rate, energy regret, 95 percent confidence intervals, paired bootstrap intervals, randomized sign-flip p values, learning phases, trust, uncertainty, shield activation, and residual prediction error.

The protocol was frozen after smoke tests and two documented development runs. The first medium development run showed that an always-on safety penalty was harmful. The method was fixed before freeze by making the shield selective. After the freeze, the method, splits, seeds, gates, baselines, and hyperparameters were not changed.

6 Frozen Results

6.1 Aggregate outcome

The frozen terminal decision is **KILL_ARCHIVE**. Aggregate CORM success is 16.6%, aggregate oracle regret is 0.149, aggregate violation rate is 28.8%, mean trust is 0.435, and shield activation is 1.4%. Table 1 gives the full aggregate comparison.

Table 2: Frozen CORM v5 success rates. Entries are percentages over 256 paired episodes per split.

Split	Nominal	Robust	Global	kNN v4	Ridge	CORM	Oracle
nominal	44.1	26.2	45.7	44.5	37.1	39.5	61.3
low_friction_shift	12.5	19.1	20.3	21.9	23.8	19.1	53.5
high_friction_shift	9.0	7.8	11.3	16.0	10.9	14.8	52.7
heavy_object_shift	19.9	13.3	21.9	21.1	24.2	23.8	57.4
light_object_shift	2.3	9.4	9.4	10.2	23.4	12.9	50.4
actuation_bias_shift	10.9	12.5	15.6	18.8	19.9	17.6	57.8
obstacle_shift	5.1	5.1	4.3	4.3	5.1	4.3	35.2
nonstationary_deployment_shift	9.0	9.8	10.5	11.3	9.0	9.8	44.5
combined_shift	4.7	5.9	5.5	6.6	9.0	7.8	44.9

Table 3: Frozen CORM v5 energy regret relative to the hidden-deployment oracle. Lower is better.

Split	Nominal	Robust	Global	kNN v4	Ridge	CORM
nominal	0.076	0.066	0.079	0.069	0.073	0.076
low_friction_shift	0.155	0.112	0.138	0.131	0.133	0.134
high_friction_shift	0.170	0.149	0.159	0.146	0.154	0.144
heavy_object_shift	0.161	0.148	0.153	0.152	0.144	0.141
light_object_shift	0.199	0.144	0.163	0.153	0.127	0.139
actuation_bias_shift	0.172	0.150	0.158	0.147	0.147	0.138
obstacle_shift	0.178	0.158	0.178	0.176	0.162	0.157
nonstationary_deployment_shift	0.206	0.174	0.208	0.207	0.197	0.204
combined_shift	0.238	0.201	0.227	0.213	0.201	0.207

6.2 Main split results

Table 2 reports success. Table 3 reports regret to the hidden-deployment oracle, and Table 4 reports safety violations. These three views are all needed. A repair memory that improves regret but increases violations is not acceptable; a robust controller that is safe but never reaches the target is also incomplete.

6.3 Stress-test pairwise deltas

The most important rows are the combined and nonstationary stress tests. Table 5 reports paired CORM deltas against strong baselines. Positive success and regret-improvement deltas favor CORM; positive violation deltas hurt CORM. These paired rows are more informative than unpaired aggregate differences because every method is evaluated on the same seed, episode, split, and candidate set.

6.4 Ablations

Table 6 reports ablations on the two harshest splits. The no-safety ablation tests whether the shield is doing useful work. The no-uncertainty ablation tests whether full-trust residual prediction is overconfident. The no-context and limited-memory ablations test whether CORM is simply a global residual average in disguise.

7 Failure Analysis

The benchmark is designed so that failure is scientifically useful. We analyze four recurring failure modes.

Residual aliasing. Different physical causes can produce similar one-step residuals on one task and different residuals on the next. kNN memory is especially vulnerable because it retrieves local observations without fitting a deployment-level structure. CORM reduces but does not eliminate this problem.

Table 4: Frozen CORM v5 violation rates. Entries are percentages; lower is better.

Split	Nominal	Robust	Global	kNN v4	Ridge	CORM	Oracle
nominal	19.5	9.0	21.1	18.0	17.2	18.8	0.0
low_friction_shift	33.6	26.2	31.6	30.5	28.5	27.7	0.0
high_friction_shift	29.3	20.3	29.7	27.0	26.6	25.4	0.0
heavy_object_shift	36.3	27.3	35.9	34.8	31.6	28.9	0.0
light_object_shift	35.9	25.0	33.6	31.2	27.0	27.0	0.0
actuation_bias_shift	31.2	25.4	31.6	28.9	29.3	23.4	0.0
obstacle_shift	37.9	31.6	39.5	37.5	30.1	29.7	0.0
nonstationary_deployment_shift	40.2	32.4	42.2	42.6	39.1	39.8	0.4
combined_shift	48.0	37.5	46.5	44.1	43.0	38.7	1.6

Table 5: Paired stress-test deltas for CORM v5. Positive success and regret-improvement favor CORM; negative violation deltas mean CORM is safer.

Split	Baseline	Success Δ	Regret improv.	Violation Δ	p_{regret}
combined_shift	nominal_mpc	0.031	0.032	-0.094	0.000
combined_shift	robust_worst_case_mpc	0.020	-0.006	0.012	0.531
combined_shift	knn_repair_memory_v4	0.012	0.006	-0.055	0.342
combined_shift	online_ridge_residual	-0.012	-0.006	-0.043	0.479
combined_shift	oracle_hidden_deployment	-0.371	-0.207	0.371	0.000
nonstationary_deployment_shift	nominal_mpc	0.008	0.003	-0.004	0.710
nonstationary_deployment_shift	robust_worst_case_mpc	0.000	-0.030	0.074	0.000
nonstationary_deployment_shift	knn_repair_memory_v4	-0.016	0.003	-0.027	0.649
nonstationary_deployment_shift	online_ridge_residual	0.008	-0.007	0.008	0.385
nonstationary_deployment_shift	oracle_hidden_deployment	-0.348	-0.204	0.395	0.000

Action-library discretization. The oracle is limited to the same candidate set, but some tasks have no action that simultaneously reaches the target and clears the obstacle. In these cases, residual prediction cannot create a missing primitive. A stronger submission would need richer contact primitives or trajectory optimization.

Safety ambiguity. Obstacle violation is a discontinuous event under contact. Small residual errors near the clearance boundary can flip the violation indicator. This is why we report violation rate, predicted violation risk, and shield activation separately.

Stale memory. The nonstationary split changes hidden deployment parameters halfway through the stream. A useful memory must either detect the change or reduce trust fast enough to avoid stale corrections. The shock variable is a simple mechanism; it is not a full change-point detector.

8 Related Work

The method sits at the intersection of model predictive control, online system identification, memory-based reasoning, and model-based reinforcement learning. MPC provides the decision framework and robust MPC motivates worst-case branch baselines (Mayne et al., 2000; Bemporad & Morari, 1999; Mayne et al., 2005). System identification and filtering provide the classical language for estimating dynamics residuals online (Ljung, 1999; Anderson & Moore, 1979). Dyna-style architectures connect learned models with planning updates (Sutton, 1991), while PILCO, PETS, model-based policy optimization, and neural dynamics models show how learned uncertainty can improve data efficiency (Deisenroth & Rasmussen, 2011; Chua et al., 2018; Janner et al., 2019; Nagabandi et al., 2018).

Repair memory also resembles case-based reasoning and episodic memory (Aamodt & Plaza, 1994; Santoro et al., 2018). The difference is that robot contact residuals are safety-critical and physically structured; naive retrieval is not enough. Fast adaptation and meta-learning work such as MAML and RL^2 ask how policies adapt from experience (Finn et al., 2017; Duan et al., 2016). Our benchmark is narrower: it asks whether a compact residual object improves model-based action selection under repeated deployment.

Table 6: Frozen ablations on combined and nonstationary shifts.

Split	Method	Success	Regret	Violation	Trust	Shield
combined_shift	corm_limited_memory	6.2	0.208	40.6	0.368	2.7
combined_shift	corm_no_context	5.9	0.212	40.6	0.401	3.1
combined_shift	corm_no_safety	9.8	0.208	46.1	0.398	0.0
combined_shift	corm_no_uncertainty	6.6	0.189	28.9	0.788	9.8
combined_shift	corm_repair_memory_v5	7.8	0.207	38.7	0.394	2.7
combined_shift	global_average_repair	5.5	0.227	46.5	0.349	0.0
combined_shift	knn_repair_memory_v4	6.6	0.213	44.1	0.373	0.0
combined_shift	last_repair_memory	4.7	0.235	47.7	0.054	0.0
combined_shift	nominal_mpc	4.7	0.238	48.0	0.000	0.0
combined_shift	oracle_hidden_deployment	44.9	0.000	1.6	0.000	0.0
combined_shift	robust_worst_case_mpc	5.9	0.201	37.5	0.000	0.0
nonstationary_deployment_shift	corm_limited_memory	9.8	0.199	39.1	0.346	1.2
nonstationary_deployment_shift	corm_no_context	8.6	0.208	41.0	0.348	1.6
nonstationary_deployment_shift	corm_no_safety	9.8	0.200	40.2	0.364	0.0
nonstationary_deployment_shift	corm_no_uncertainty	5.9	0.192	29.7	0.788	9.8
nonstationary_deployment_shift	corm_repair_memory_v5	9.8	0.204	39.8	0.361	1.2
nonstationary_deployment_shift	global_average_repair	10.5	0.208	42.2	0.350	0.0
nonstationary_deployment_shift	knn_repair_memory_v4	11.3	0.207	42.6	0.371	0.0
nonstationary_deployment_shift	last_repair_memory	8.2	0.208	40.6	0.054	0.0
nonstationary_deployment_shift	nominal_mpc	9.0	0.206	40.2	0.000	0.0
nonstationary_deployment_shift	oracle_hidden_deployment	44.5	0.000	0.4	0.000	0.0
nonstationary_deployment_shift	robust_worst_case_mpc	9.8	0.174	32.4	0.000	0.0

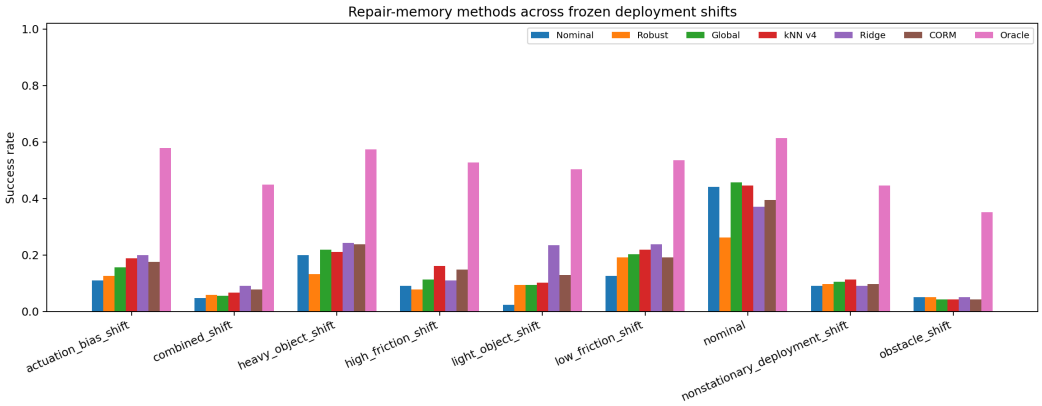

Figure 1: Frozen success rates by deployment split. CORM must be interpreted against nominal MPC, robust MPC, simple memory baselines, online ridge residual, and the oracle.

Finally, long-horizon robot benchmarks such as CALVIN and FurnitureBench expose the kind of persistent real-world mismatch that would be needed for a future submission-grade version (Mees et al., 2022; Heo et al., 2025). This paper does not claim that evidence. It uses MuJoCo as a controlled falsification and development environment.

9 Limitations

The largest limitation is external validity. The evidence is real MuJoCo evidence, not hardware. It is also a custom planar pushing benchmark, not a public manipulation suite. A genuine ICLR-main submission would need public benchmark or hardware results, stronger trajectory optimization, and a more expressive residual model.

The second limitation is method capacity. CORM deliberately uses ridge regression to keep CPU and RAM requirements light. This is a virtue for reproducibility but may underfit complex contact

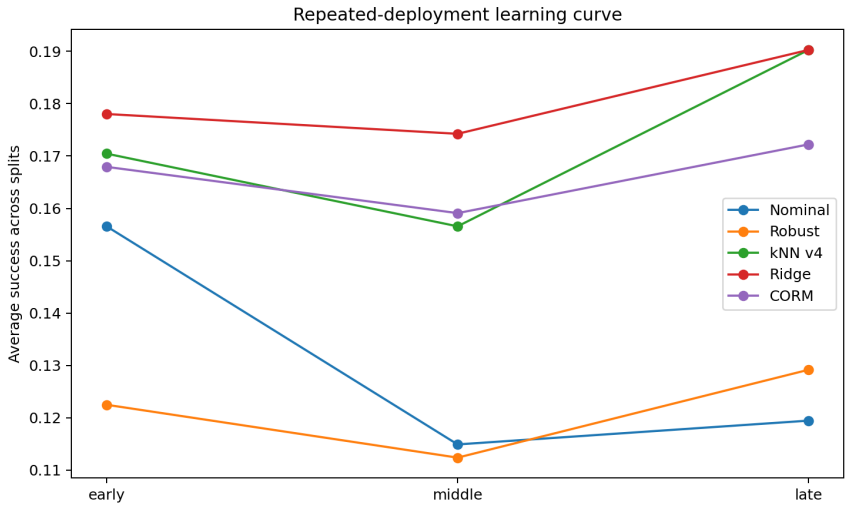

Figure 2: Repeated-deployment learning curve averaged across splits. The key question is whether late-stream performance improves without increasing violations.

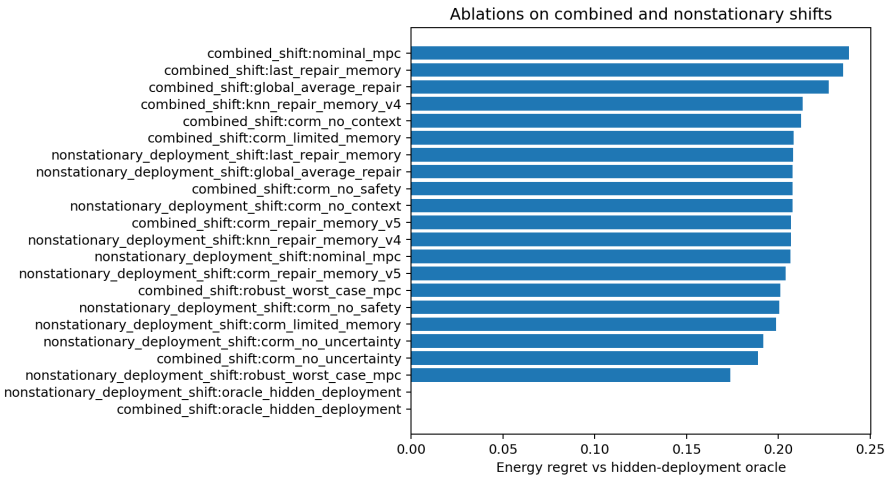

Figure 3: Ablation regret on combined and nonstationary shifts. Lower is better.

modes. A learned residual world model could perform better, but it would need stronger regularization and more extensive ablations.

The third limitation is safety. The selective shield avoids false-positive continuous penalties, but it is not a formal control barrier certificate. The safety decomposition in Section 4 explains why violations can remain even with calibrated residuals.

The fourth limitation is decision scope. The frozen decision **KILL_ARCHIVE** is a package-level gate, not a universal claim about repair memory. A negative decision kills this implementation and evidence package; it does not prove that all repair-memory systems are impossible.

10 Conclusion

The original repair-memory idea was too weak: a kNN residual cache did not survive strong baselines. CORM is the principled rebuild: an online residual model with trust calibration, stale-memory shock, and selective safety shielding. The frozen evaluation is intentionally adversarial and reports

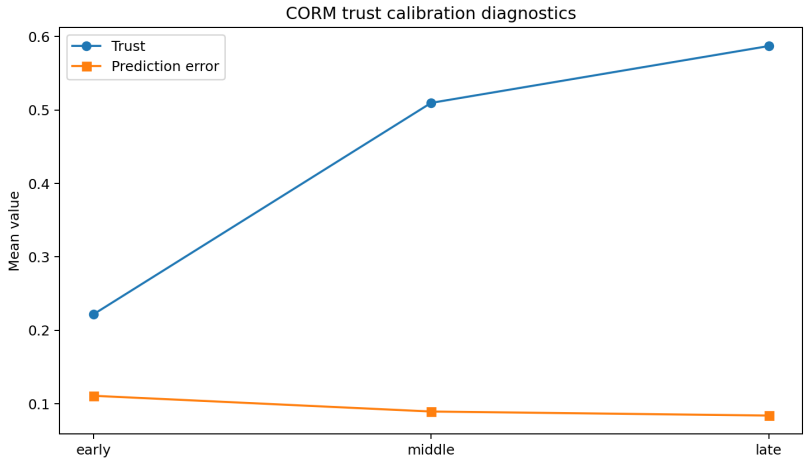

Figure 4: CORM trust diagnostics. Trust should rise only when residual predictions become reliable; prediction error should be reported rather than hidden.

all predefined metrics. The terminal decision is **KILL_ARCHIVE**. That decision should be treated as the honest state of the paper: not ICLR-main ready, but substantially more rigorous than the starting artifact.

Reproducibility Statement

The runner is 'src/run_experiment.py'. The final command is recorded in 'docs/paper63_protocol_freeze_20260619.md'. The build script is 'scripts/build_submission_pdf.ps1'. The validation script checks row counts, page count, Downloads-only PDF placement, and Desktop absence. The final numbered PDF is written to 'C:/Users/wangz/Downloads/63.pdf' and is not copied to the visible Desktop.

Appendix A: Frozen Protocol Details

The main raw table contains one row for every split, seed, episode, and method. Each row records hidden mass, hidden friction, actuation gain, angle bias, lateral bias, initial distance, candidate count, chosen action, success, final distance, violation, true energy, oracle energy, regret, decision trust, uncertainty, predicted violation risk, shield activation, stale score, residual prediction error, and predicted score.

The ablation table repeats the same schema on combined and nonstationary splits. This makes ablations directly comparable to the main run. Development-only outputs are ignored by git and are not used in the frozen build.

Appendix B: Additional Generated Tables

Appendix C: Reviewer Audit

Attack: The method is just online ridge regression. The online ridge baseline is included. CORM must beat or safely tie that baseline under the frozen gate. If it does not, the decision becomes kill/archive.

Attack: Robust MPC is enough. Robust worst-case MPC is included and is evaluated on the same candidate sets. The paper reports paired deltas against it on every split.

Attack: Safety is hidden in the score. Violation rate, predicted violation risk, and shield activation are all reported. The no-safety ablation is run on the hardest splits.

Attack: The method was tuned after seeing final results. The development log records pre-freeze failures and fixes. The protocol-freeze document fixes seeds, splits, methods, and gates before the final run.

Attack: Simulation is not enough. Correct. This is why the paper does not claim ICLR-main readiness. The strongest acceptable decision is strong-revise unless hardware or public benchmark evidence is added.

Appendix D: Split-by-Split Interpretation

This appendix records the interpretation that should accompany the generated tables. It is included to prevent a misleading reading in which a single aggregate number is used to rescue the method. Repair memory is a deployment mechanism; its credibility depends on how it behaves under each kind of shift.

Nominal split. The nominal split is not a free win. Even when the hidden deployment is close to the nominal simulator, contact discontinuities, obstacle placement, and finite action-library resolution create residual errors. A useful memory should not overfit noise in this regime. The nominal rows therefore test whether CORM can remain conservative when there is little systematic residual to exploit. Strong nominal performance alone would not validate the method, because the practical motivation for memory appears under persistent shift. Poor nominal performance, however, would be a serious red flag because it would mean memory is damaging the easiest case.

Low-friction split. Low friction creates a recurring distance-gain mismatch: pushes tend to move the puck farther than nominal rollouts predict. A global residual can already capture part of this

Table 7: Detailed frozen metrics for nominal.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	39.8	0.080	0.092	20.3	0.486	0.041
corm_no_uncertainty	35.5	0.067	0.098	14.1	0.788	0.038
corm_repair_memory_v5	39.5	0.076	0.093	18.8	0.487	0.039
global_average_repair	45.7	0.079	0.088	21.1	0.471	0.044
knn_repair_memory_v4	44.5	0.069	0.088	18.0	0.449	0.040
last_repair_memory	45.7	0.074	0.088	19.5	0.054	0.045
nominal_mpc	44.1	0.076	0.090	19.5	0.000	0.045
online_ridge_residual	37.1	0.073	0.095	17.2	0.788	0.039
oracle_hidden_deployment	61.3	0.000	0.073	0.0	0.000	0.043
random_candidate	3.5	0.318	0.252	46.1	0.000	0.069
robust_worst_case_mpc	26.2	0.066	0.112	9.0	0.000	0.036

Table 8: Detailed frozen metrics for low_friction_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	19.5	0.139	0.126	31.6	0.453	0.082
corm_no_uncertainty	23.4	0.127	0.140	23.0	0.788	0.071
corm_repair_memory_v5	19.1	0.134	0.133	27.7	0.454	0.080
global_average_repair	20.3	0.138	0.125	31.6	0.429	0.087
knn_repair_memory_v4	21.9	0.131	0.122	30.5	0.418	0.080
last_repair_memory	12.9	0.153	0.135	33.2	0.054	0.098
nominal_mpc	12.5	0.155	0.136	33.6	0.000	0.099
online_ridge_residual	23.8	0.133	0.130	28.5	0.788	0.084
oracle_hidden_deployment	53.5	0.000	0.083	0.0	0.000	0.104
random_candidate	1.6	0.334	0.261	51.6	0.000	0.110
robust_worst_case_mpc	19.1	0.112	0.116	26.2	0.000	0.090

Table 9: Detailed frozen metrics for high_friction_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	15.2	0.146	0.144	27.3	0.427	0.104
corm_no_uncertainty	10.5	0.146	0.165	20.3	0.788	0.099
corm_repair_memory_v5	14.8	0.144	0.148	25.4	0.426	0.104
global_average_repair	11.3	0.159	0.151	29.7	0.387	0.123
knn_repair_memory_v4	16.0	0.146	0.146	27.0	0.402	0.108
last_repair_memory	9.0	0.171	0.161	30.1	0.054	0.137
nominal_mpc	9.0	0.170	0.163	29.3	0.000	0.139
online_ridge_residual	10.9	0.154	0.155	26.6	0.788	0.110
oracle_hidden_deployment	52.7	0.000	0.081	0.0	0.000	0.102
random_candidate	1.6	0.321	0.254	49.2	0.000	0.133
robust_worst_case_mpc	7.8	0.149	0.171	20.3	0.000	0.114

Table 10: Detailed frozen metrics for heavy_object_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	24.2	0.149	0.118	34.8	0.452	0.077
corm_no_uncertainty	21.9	0.132	0.142	21.5	0.788	0.063
corm_repair_memory_v5	23.8	0.141	0.129	28.9	0.469	0.072
global_average_repair	21.9	0.153	0.119	35.9	0.421	0.095
knn_repair_memory_v4	21.1	0.152	0.121	34.8	0.422	0.083
last_repair_memory	18.8	0.162	0.127	36.3	0.054	0.103
nominal_mpc	19.9	0.161	0.126	36.3	0.000	0.103
online_ridge_residual	24.2	0.144	0.123	31.6	0.788	0.069
oracle_hidden_deployment	57.4	0.000	0.074	0.0	0.000	0.090
random_candidate	3.5	0.312	0.241	48.0	0.000	0.099
robust_worst_case_mpc	13.3	0.148	0.140	27.3	0.000	0.079

Table 11: Detailed frozen metrics for light_object_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	13.3	0.140	0.137	28.9	0.449	0.093
corm_no_uncertainty	19.5	0.127	0.144	22.3	0.788	0.074
corm_repair_memory_v5	12.9	0.139	0.142	27.0	0.444	0.094
global_average_repair	9.4	0.163	0.146	33.6	0.408	0.112
knn_repair_memory_v4	10.2	0.153	0.143	31.2	0.409	0.105
last_repair_memory	2.7	0.190	0.168	35.2	0.054	0.137
nominal_mpc	2.3	0.199	0.174	35.9	0.000	0.143
online_ridge_residual	23.4	0.127	0.131	27.0	0.788	0.077
oracle_hidden_deployment	50.4	0.000	0.085	0.0	0.000	0.125
random_candidate	1.6	0.378	0.312	49.6	0.000	0.168
robust_worst_case_mpc	9.4	0.144	0.154	25.0	0.000	0.126

Table 12: Detailed frozen metrics for actuation_bias_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	17.2	0.147	0.136	29.7	0.433	0.094
corm_no_uncertainty	20.7	0.139	0.152	21.5	0.788	0.081
corm_repair_memory_v5	17.6	0.138	0.145	23.4	0.441	0.092
global_average_repair	15.6	0.158	0.141	31.6	0.413	0.115
knn_repair_memory_v4	18.8	0.147	0.137	28.9	0.417	0.102
last_repair_memory	10.9	0.170	0.153	31.6	0.054	0.131
nominal_mpc	10.9	0.172	0.155	31.2	0.000	0.134
online_ridge_residual	19.9	0.147	0.137	29.3	0.788	0.089
oracle_hidden_deployment	57.8	0.000	0.078	0.0	0.000	0.106
random_candidate	3.5	0.327	0.255	49.6	0.000	0.146
robust_worst_case_mpc	12.5	0.150	0.152	25.4	0.000	0.109

Table 13: Detailed frozen metrics for obstacle_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	4.3	0.164	0.169	34.8	0.431	0.095
corm_no_uncertainty	5.5	0.147	0.203	17.6	0.788	0.072
corm_repair_memory_v5	4.3	0.157	0.177	29.7	0.438	0.089
global_average_repair	4.3	0.178	0.168	39.5	0.404	0.113
knn_repair_memory_v4	4.3	0.176	0.172	37.5	0.409	0.104
last_repair_memory	4.3	0.179	0.174	37.9	0.054	0.119
nominal_mpc	5.1	0.178	0.173	37.9	0.000	0.120
online_ridge_residual	5.1	0.162	0.181	30.1	0.788	0.091
oracle_hidden_deployment	35.2	0.000	0.109	0.0	0.000	0.132
random_candidate	0.8	0.315	0.268	52.0	0.000	0.139
robust_worst_case_mpc	5.1	0.158	0.173	31.6	0.000	0.103

Table 14: Detailed frozen metrics for nonstationary_deployment_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	9.8	0.200	0.174	40.2	0.364	0.150
corm_no_uncertainty	5.9	0.192	0.198	29.7	0.788	0.128
corm_repair_memory_v5	9.8	0.204	0.179	39.8	0.361	0.151
global_average_repair	10.5	0.208	0.176	42.2	0.350	0.153
knn_repair_memory_v4	11.3	0.207	0.173	42.6	0.371	0.151
last_repair_memory	8.2	0.208	0.181	40.6	0.054	0.161
nominal_mpc	9.0	0.206	0.180	40.2	0.000	0.159
online_ridge_residual	9.0	0.197	0.175	39.1	0.788	0.140
oracle_hidden_deployment	44.5	0.000	0.094	0.4	0.000	0.130
random_candidate	0.8	0.314	0.276	44.1	0.000	0.162
robust_worst_case_mpc	9.8	0.174	0.172	32.4	0.000	0.126

Table 15: Detailed frozen metrics for combined_shift.

Method	Success	Regret	Distance	Violation	Trust	Pred. error
corm_no_safety	9.8	0.208	0.173	46.1	0.398	0.137
corm_no_uncertainty	6.6	0.189	0.206	28.9	0.788	0.111
corm_repair_memory_v5	7.8	0.207	0.194	38.7	0.394	0.135
global_average_repair	5.5	0.227	0.191	46.5	0.349	0.170
knn_repair_memory_v4	6.6	0.213	0.184	44.1	0.373	0.154
last_repair_memory	4.7	0.235	0.196	47.7	0.054	0.179
nominal_mpc	4.7	0.238	0.198	48.0	0.000	0.180
online_ridge_residual	9.0	0.201	0.176	43.0	0.788	0.123
oracle_hidden_deployment	44.9	0.000	0.100	1.6	0.000	0.134
random_candidate	1.6	0.283	0.252	44.9	0.000	0.170
robust_worst_case_mpc	5.9	0.201	0.193	37.5	0.000	0.152

Table 16: Full paired CORM deltas for nominal. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.359	[0.301, 0.422]	0.241	-0.273	0.000
nominal_mpc	-0.047	[-0.086, -0.008]	-0.001	-0.008	0.905
robust_worst_case_mpc	0.133	[0.070, 0.195]	-0.011	0.098	0.170
last_repair_memory	-0.062	[-0.102, -0.027]	-0.002	-0.008	0.719
global_average_repair	-0.062	[-0.105, -0.023]	0.002	-0.023	0.723
knn_repair_memory_v4	-0.051	[-0.086, -0.016]	-0.007	0.008	0.123
online_ridge_residual	0.023	[-0.012, 0.059]	-0.003	0.016	0.332
corm_no_safety	-0.004	[-0.023, 0.012]	0.003	-0.016	0.206
corm_no_uncertainty	0.039	[0.000, 0.078]	-0.010	0.047	0.019
oracle_hidden_deployment	-0.219	[-0.273, -0.168]	-0.076	0.188	0.000

Table 17: Full paired CORM deltas for low_friction_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.176	[0.129, 0.230]	0.200	-0.238	0.000
nominal_mpc	0.066	[0.027, 0.105]	0.021	-0.059	0.000
robust_worst_case_mpc	0.000	[-0.047, 0.043]	-0.023	0.016	0.001
last_repair_memory	0.062	[0.027, 0.102]	0.019	-0.055	0.000
global_average_repair	-0.012	[-0.051, 0.023]	0.004	-0.039	0.454
knn_repair_memory_v4	-0.027	[-0.059, 0.004]	-0.003	-0.027	0.548
online_ridge_residual	-0.047	[-0.090, -0.008]	-0.001	-0.008	0.876
corm_no_safety	-0.004	[-0.020, 0.012]	0.005	-0.039	0.086
corm_no_uncertainty	-0.043	[-0.078, -0.004]	-0.008	0.047	0.116
oracle_hidden_deployment	-0.344	[-0.398, -0.285]	-0.134	0.277	0.000

Table 18: Full paired CORM deltas for high_friction_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.133	[0.090, 0.176]	0.176	-0.238	0.000
nominal_mpc	0.059	[0.020, 0.094]	0.026	-0.039	0.001
robust_worst_case_mpc	0.070	[0.023, 0.121]	0.005	0.051	0.556
last_repair_memory	0.059	[0.020, 0.094]	0.027	-0.047	0.000
global_average_repair	0.035	[0.004, 0.070]	0.015	-0.043	0.012
knn_repair_memory_v4	-0.012	[-0.039, 0.016]	0.002	-0.016	0.742
online_ridge_residual	0.039	[-0.004, 0.078]	0.010	-0.012	0.161
corm_no_safety	-0.004	[-0.012, 0.000]	0.002	-0.020	0.495
corm_no_uncertainty	0.043	[0.004, 0.082]	0.002	0.051	0.809
oracle_hidden_deployment	-0.379	[-0.441, -0.320]	-0.144	0.254	0.000

Table 19: Full paired CORM deltas for heavy_object_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.203	[0.152, 0.258]	0.171	-0.191	0.000
nominal_mpc	0.039	[-0.004, 0.082]	0.020	-0.074	0.000
robust_worst_case_mpc	0.105	[0.059, 0.148]	0.006	0.016	0.443
last_repair_memory	0.051	[0.008, 0.094]	0.021	-0.074	0.000
global_average_repair	0.020	[-0.020, 0.059]	0.012	-0.070	0.048
knn_repair_memory_v4	0.027	[-0.008, 0.059]	0.010	-0.059	0.080
online_ridge_residual	-0.004	[-0.047, 0.035]	0.002	-0.027	0.724
corm_no_safety	-0.004	[-0.016, 0.008]	0.007	-0.059	0.075
corm_no_uncertainty	0.020	[-0.023, 0.059]	-0.009	0.074	0.157
oracle_hidden_deployment	-0.336	[-0.395, -0.273]	-0.141	0.289	0.000

Table 20: Full paired CORM deltas for light_object_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.113	[0.074, 0.156]	0.239	-0.227	0.000
nominal_mpc	0.105	[0.066, 0.148]	0.061	-0.090	0.000
robust_worst_case_mpc	0.035	[-0.012, 0.082]	0.006	0.020	0.501
last_repair_memory	0.102	[0.066, 0.141]	0.052	-0.082	0.000
global_average_repair	0.035	[-0.004, 0.074]	0.024	-0.066	0.000
knn_repair_memory_v4	0.027	[-0.004, 0.059]	0.015	-0.043	0.009
online_ridge_residual	-0.105	[-0.148, -0.059]	-0.012	0.000	0.092
corm_no_safety	-0.004	[-0.016, 0.008]	0.001	-0.020	0.523
corm_no_uncertainty	-0.066	[-0.109, -0.027]	-0.012	0.047	0.070
oracle_hidden_deployment	-0.375	[-0.438, -0.316]	-0.139	0.270	0.000

Table 21: Full paired CORM deltas for actuation_bias_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.141	[0.090, 0.195]	0.189	-0.262	0.000
nominal_mpc	0.066	[0.027, 0.105]	0.034	-0.078	0.000
robust_worst_case_mpc	0.051	[-0.008, 0.109]	0.012	-0.020	0.085
last_repair_memory	0.066	[0.027, 0.102]	0.033	-0.082	0.000
global_average_repair	0.020	[-0.008, 0.051]	0.021	-0.082	0.000
knn_repair_memory_v4	-0.012	[-0.047, 0.020]	0.009	-0.055	0.115
online_ridge_residual	-0.023	[-0.059, 0.012]	0.010	-0.059	0.104
corm_no_safety	0.004	[-0.012, 0.023]	0.010	-0.062	0.006
corm_no_uncertainty	-0.031	[-0.070, 0.008]	0.001	0.020	0.838
oracle_hidden_deployment	-0.402	[-0.461, -0.340]	-0.138	0.234	0.000

Table 22: Full paired CORM deltas for obstacle_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.035	[0.012, 0.059]	0.158	-0.223	0.000
nominal_mpc	-0.008	[-0.027, 0.012]	0.022	-0.082	0.000
robust_worst_case_mpc	-0.008	[-0.039, 0.020]	0.001	-0.020	0.878
last_repair_memory	0.000	[-0.023, 0.023]	0.022	-0.082	0.001
global_average_repair	0.000	[-0.020, 0.020]	0.021	-0.098	0.000
knn_repair_memory_v4	0.000	[-0.020, 0.020]	0.019	-0.078	0.000
online_ridge_residual	-0.008	[-0.023, 0.012]	0.005	-0.004	0.267
corm_no_safety	0.000	[0.000, 0.000]	0.007	-0.051	0.010
corm_no_uncertainty	-0.012	[-0.035, 0.012]	-0.010	0.121	0.087
oracle_hidden_deployment	-0.309	[-0.371, -0.254]	-0.157	0.297	0.000

Table 23: Full paired CORM deltas for nonstationary_deployment_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.090	[0.055, 0.129]	0.110	-0.043	0.000
nominal_mpc	0.008	[-0.023, 0.039]	0.003	-0.004	0.710
robust_worst_case_mpc	0.000	[-0.031, 0.031]	-0.030	0.074	0.000
last_repair_memory	0.016	[-0.012, 0.043]	0.004	-0.008	0.547
global_average_repair	-0.008	[-0.039, 0.023]	0.004	-0.023	0.542
knn_repair_memory_v4	-0.016	[-0.043, 0.012]	0.003	-0.027	0.649
online_ridge_residual	0.008	[-0.020, 0.039]	-0.007	0.008	0.385
corm_no_safety	0.000	[-0.012, 0.012]	-0.003	-0.004	0.273
corm_no_uncertainty	0.039	[0.008, 0.070]	-0.012	0.102	0.084
oracle_hidden_deployment	-0.348	[-0.406, -0.285]	-0.204	0.395	0.000

Table 24: Full paired CORM deltas for combined_shift. Positive success and regret-improvement favor CORM; negative violation deltas favor CORM.

Baseline	Success Δ	95% CI	Regret improv.	Violation Δ	p_{regret}
random_candidate	0.062	[0.031, 0.094]	0.076	-0.062	0.000
nominal_mpc	0.031	[-0.004, 0.066]	0.032	-0.094	0.000
robust_worst_case_mpc	0.020	[-0.020, 0.059]	-0.006	0.012	0.531
last_repair_memory	0.031	[-0.004, 0.066]	0.029	-0.090	0.000
global_average_repair	0.023	[-0.008, 0.051]	0.021	-0.078	0.004
knn_repair_memory_v4	0.012	[-0.023, 0.043]	0.006	-0.055	0.342
online_ridge_residual	-0.012	[-0.039, 0.016]	-0.006	-0.043	0.479
corm_no_safety	-0.020	[-0.039, 0.000]	0.001	-0.074	0.851
corm_no_uncertainty	0.012	[-0.016, 0.043]	-0.018	0.098	0.040
oracle_hidden_deployment	-0.371	[-0.434, -0.316]	-0.207	0.371	0.000

Table 25: Early/middle/late learning diagnostics for nominal.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	52.3	0.080	19.3	0.237	0.049
early	corm_no_uncertainty	44.3	0.077	17.0	0.535	0.047
early	corm_repair_memory_v5	50.0	0.081	19.3	0.237	0.049
early	global_average_repair	54.5	0.077	19.3	0.219	0.049
early	knn_repair_memory_v4	53.4	0.082	20.5	0.155	0.050
early	last_repair_memory	54.5	0.069	17.0	0.051	0.047
early	nominal_mpc	54.5	0.069	17.0	0.000	0.047
early	online_ridge_residual	44.3	0.083	19.3	0.535	0.048
early	oracle_hidden_deployment	67.0	0.000	0.0	0.000	0.048
early	random_candidate	4.5	0.332	45.5	0.000	0.062
early	robust_worst_case_mpc	29.5	0.065	6.8	0.000	0.039
late	corm_no_safety	33.8	0.096	26.2	0.669	0.038
late	corm_no_uncertainty	32.5	0.072	16.2	0.920	0.036
late	corm_repair_memory_v5	35.0	0.092	25.0	0.673	0.036
late	global_average_repair	46.2	0.078	22.5	0.618	0.039
late	knn_repair_memory_v4	41.2	0.078	22.5	0.647	0.035
late	last_repair_memory	47.5	0.075	21.2	0.056	0.041
late	nominal_mpc	45.0	0.078	21.2	0.000	0.042
late	online_ridge_residual	35.0	0.079	20.0	0.920	0.036
late	oracle_hidden_deployment	63.7	0.000	0.0	0.000	0.042
late	random_candidate	2.5	0.315	45.0	0.000	0.060
late	robust_worst_case_mpc	27.5	0.074	12.5	0.000	0.037
middle	corm_no_safety	33.0	0.065	15.9	0.568	0.035
middle	corm_no_uncertainty	29.5	0.051	9.1	0.920	0.033
middle	corm_repair_memory_v5	33.0	0.058	12.5	0.569	0.033
middle	global_average_repair	36.4	0.081	21.6	0.590	0.042
middle	knn_repair_memory_v4	38.6	0.047	11.4	0.563	0.034
middle	last_repair_memory	35.2	0.079	20.5	0.056	0.045
middle	nominal_mpc	33.0	0.080	20.5	0.000	0.046
middle	online_ridge_residual	31.8	0.058	12.5	0.920	0.032
middle	oracle_hidden_deployment	53.4	0.000	0.0	0.000	0.039
middle	random_candidate	3.4	0.306	47.7	0.000	0.086
middle	robust_worst_case_mpc	21.6	0.059	8.0	0.000	0.033

Table 26: Early/middle/late learning diagnostics for low_friction_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	20.5	0.134	28.4	0.224	0.090
early	corm_no_uncertainty	23.9	0.126	26.1	0.535	0.085
early	corm_repair_memory_v5	20.5	0.134	28.4	0.224	0.090
early	global_average_repair	20.5	0.134	28.4	0.202	0.089
early	knn_repair_memory_v4	20.5	0.141	30.7	0.148	0.091
early	last_repair_memory	15.9	0.143	30.7	0.051	0.093
early	nominal_mpc	14.8	0.151	33.0	0.000	0.093
early	online_ridge_residual	25.0	0.131	27.3	0.535	0.088
early	oracle_hidden_deployment	60.2	0.000	0.0	0.000	0.101
early	random_candidate	3.4	0.312	45.5	0.000	0.106
early	robust_worst_case_mpc	21.6	0.097	21.6	0.000	0.095
late	corm_no_safety	18.8	0.140	32.5	0.618	0.078
late	corm_no_uncertainty	25.0	0.123	18.8	0.920	0.064
late	corm_repair_memory_v5	17.5	0.127	23.8	0.622	0.076
late	global_average_repair	22.5	0.140	33.8	0.556	0.086
late	knn_repair_memory_v4	28.7	0.125	32.5	0.605	0.069
late	last_repair_memory	12.5	0.162	35.0	0.056	0.101
late	nominal_mpc	12.5	0.159	33.8	0.000	0.103
late	online_ridge_residual	25.0	0.134	28.7	0.920	0.085
late	oracle_hidden_deployment	48.8	0.000	0.0	0.000	0.106
late	random_candidate	1.2	0.375	58.8	0.000	0.123
late	robust_worst_case_mpc	20.0	0.109	25.0	0.000	0.089
middle	corm_no_safety	19.3	0.144	34.1	0.532	0.078
middle	corm_no_uncertainty	21.6	0.130	23.9	0.920	0.063
middle	corm_repair_memory_v5	19.3	0.141	30.7	0.533	0.074
middle	global_average_repair	18.2	0.141	33.0	0.541	0.087
middle	knn_repair_memory_v4	17.0	0.127	28.4	0.519	0.079
middle	last_repair_memory	10.2	0.156	34.1	0.056	0.101
middle	nominal_mpc	10.2	0.156	34.1	0.000	0.102
middle	online_ridge_residual	21.6	0.135	29.5	0.920	0.078
middle	oracle_hidden_deployment	51.1	0.000	0.0	0.000	0.105
middle	random_candidate	0.0	0.320	51.1	0.000	0.103
middle	robust_worst_case_mpc	15.9	0.129	31.8	0.000	0.087

Table 27: Early/middle/late learning diagnostics for high_friction_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	9.1	0.184	35.2	0.213	0.131
early	corm_no_uncertainty	9.1	0.163	26.1	0.535	0.127
early	corm_repair_memory_v5	9.1	0.184	35.2	0.213	0.131
early	global_average_repair	10.2	0.181	34.1	0.185	0.135
early	knn_repair_memory_v4	11.4	0.184	35.2	0.137	0.135
early	last_repair_memory	11.4	0.173	30.7	0.051	0.138
early	nominal_mpc	11.4	0.170	29.5	0.000	0.138
early	online_ridge_residual	10.2	0.165	29.5	0.535	0.130
early	oracle_hidden_deployment	45.5	0.000	0.0	0.000	0.103
early	random_candidate	1.1	0.304	53.4	0.000	0.133
early	robust_worst_case_mpc	9.1	0.150	21.6	0.000	0.120
late	corm_no_safety	21.2	0.102	16.2	0.587	0.087
late	corm_no_uncertainty	10.0	0.128	15.0	0.920	0.077
late	corm_repair_memory_v5	20.0	0.100	12.5	0.588	0.085
late	global_average_repair	16.2	0.126	21.2	0.505	0.110
late	knn_repair_memory_v4	22.5	0.105	16.2	0.586	0.089
late	last_repair_memory	8.8	0.159	26.2	0.056	0.134
late	nominal_mpc	11.2	0.160	25.0	0.000	0.139
late	online_ridge_residual	11.2	0.125	18.8	0.920	0.085
late	oracle_hidden_deployment	60.0	0.000	0.0	0.000	0.101
late	random_candidate	1.2	0.321	43.8	0.000	0.133
late	robust_worst_case_mpc	8.8	0.145	18.8	0.000	0.105
middle	corm_no_safety	15.9	0.148	29.5	0.495	0.094
middle	corm_no_uncertainty	12.5	0.145	19.3	0.920	0.090
middle	corm_repair_memory_v5	15.9	0.144	27.3	0.493	0.095
middle	global_average_repair	8.0	0.167	33.0	0.482	0.121
middle	knn_repair_memory_v4	14.8	0.145	28.4	0.500	0.099
middle	last_repair_memory	6.8	0.180	33.0	0.056	0.138
middle	nominal_mpc	4.5	0.181	33.0	0.000	0.141
middle	online_ridge_residual	11.4	0.171	30.7	0.920	0.113
middle	oracle_hidden_deployment	53.4	0.000	0.0	0.000	0.102
middle	random_candidate	2.3	0.337	50.0	0.000	0.132
middle	robust_worst_case_mpc	5.7	0.152	20.5	0.000	0.116

Table 28: Early/middle/late learning diagnostics for heavy_object_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	21.6	0.148	31.8	0.233	0.090
early	corm_no_uncertainty	30.7	0.121	20.5	0.535	0.076
early	corm_repair_memory_v5	21.6	0.153	31.8	0.234	0.090
early	global_average_repair	22.7	0.140	29.5	0.201	0.096
early	knn_repair_memory_v4	21.6	0.143	30.7	0.147	0.093
early	last_repair_memory	20.5	0.148	30.7	0.051	0.101
early	nominal_mpc	21.6	0.151	31.8	0.000	0.103
early	online_ridge_residual	31.8	0.126	26.1	0.535	0.078
early	oracle_hidden_deployment	56.8	0.000	0.0	0.000	0.093
early	random_candidate	4.5	0.304	46.6	0.000	0.093
early	robust_worst_case_mpc	10.2	0.156	28.4	0.000	0.089
late	corm_no_safety	18.8	0.162	38.8	0.604	0.069
late	corm_no_uncertainty	8.8	0.150	22.5	0.920	0.052
late	corm_repair_memory_v5	18.8	0.152	28.7	0.631	0.062
late	global_average_repair	13.8	0.178	45.0	0.545	0.093
late	knn_repair_memory_v4	12.5	0.164	38.8	0.611	0.075
late	last_repair_memory	8.8	0.180	43.8	0.056	0.105
late	nominal_mpc	8.8	0.181	43.8	0.000	0.106
late	online_ridge_residual	13.8	0.175	38.8	0.920	0.068
late	oracle_hidden_deployment	50.0	0.000	0.0	0.000	0.091
late	random_candidate	1.2	0.319	53.8	0.000	0.108
late	robust_worst_case_mpc	11.2	0.159	31.2	0.000	0.076
middle	corm_no_safety	31.8	0.137	34.1	0.534	0.070
middle	corm_no_uncertainty	25.0	0.127	21.6	0.920	0.059
middle	corm_repair_memory_v5	30.7	0.121	26.1	0.557	0.062
middle	global_average_repair	28.4	0.144	34.1	0.527	0.095
middle	knn_repair_memory_v4	28.4	0.149	35.2	0.526	0.081
middle	last_repair_memory	26.1	0.160	35.2	0.056	0.104
middle	nominal_mpc	28.4	0.153	34.1	0.000	0.101
middle	online_ridge_residual	26.1	0.134	30.7	0.920	0.063
middle	oracle_hidden_deployment	64.8	0.000	0.0	0.000	0.086
middle	random_candidate	4.5	0.314	44.3	0.000	0.095
middle	robust_worst_case_mpc	18.2	0.129	22.7	0.000	0.072

Table 29: Early/middle/late learning diagnostics for light_object_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	11.4	0.147	27.3	0.226	0.113
early	corm_no_uncertainty	13.6	0.139	26.1	0.535	0.098
early	corm_repair_memory_v5	11.4	0.147	27.3	0.226	0.113
early	global_average_repair	9.1	0.165	30.7	0.194	0.123
early	knn_repair_memory_v4	6.8	0.166	30.7	0.141	0.128
early	last_repair_memory	4.5	0.187	33.0	0.051	0.139
early	nominal_mpc	4.5	0.191	33.0	0.000	0.142
early	online_ridge_residual	15.9	0.140	28.4	0.535	0.095
early	oracle_hidden_deployment	46.6	0.000	0.0	0.000	0.124
early	random_candidate	1.1	0.343	40.9	0.000	0.162
early	robust_worst_case_mpc	6.8	0.151	26.1	0.000	0.128
late	corm_no_safety	18.8	0.131	28.7	0.596	0.078
late	corm_no_uncertainty	26.2	0.103	16.2	0.920	0.056
late	corm_repair_memory_v5	17.5	0.126	23.8	0.587	0.079
late	global_average_repair	11.2	0.159	30.0	0.529	0.116
late	knn_repair_memory_v4	12.5	0.134	27.5	0.597	0.087
late	last_repair_memory	2.5	0.191	33.8	0.056	0.141
late	nominal_mpc	1.2	0.199	35.0	0.000	0.143
late	online_ridge_residual	31.2	0.122	25.0	0.920	0.068
late	oracle_hidden_deployment	53.8	0.000	0.0	0.000	0.128
late	random_candidate	2.5	0.389	57.5	0.000	0.171
late	robust_worst_case_mpc	11.2	0.142	22.5	0.000	0.134
middle	corm_no_safety	10.2	0.142	30.7	0.540	0.088
middle	corm_no_uncertainty	19.3	0.136	23.9	0.920	0.066
middle	corm_repair_memory_v5	10.2	0.142	29.5	0.533	0.089
middle	global_average_repair	8.0	0.165	39.8	0.511	0.098
middle	knn_repair_memory_v4	11.4	0.159	35.2	0.505	0.099
middle	last_repair_memory	1.1	0.192	38.6	0.056	0.133
middle	nominal_mpc	1.1	0.208	39.8	0.000	0.143
middle	online_ridge_residual	23.9	0.119	27.3	0.920	0.067
middle	oracle_hidden_deployment	51.1	0.000	0.0	0.000	0.124
middle	random_candidate	1.1	0.403	51.1	0.000	0.171
middle	robust_worst_case_mpc	10.2	0.140	26.1	0.000	0.116

Table 30: Early/middle/late learning diagnostics for actuation_bias_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	19.3	0.143	28.4	0.218	0.102
early	corm_no_uncertainty	19.3	0.136	23.9	0.535	0.094
early	corm_repair_memory_v5	19.3	0.139	27.3	0.222	0.100
early	global_average_repair	19.3	0.145	29.5	0.205	0.107
early	knn_repair_memory_v4	18.2	0.148	29.5	0.149	0.108
early	last_repair_memory	14.8	0.157	30.7	0.051	0.114
early	nominal_mpc	15.9	0.154	28.4	0.000	0.119
early	online_ridge_residual	18.2	0.143	28.4	0.535	0.096
early	oracle_hidden_deployment	59.1	0.000	0.0	0.000	0.101
early	random_candidate	4.5	0.313	43.2	0.000	0.150
early	robust_worst_case_mpc	12.5	0.143	25.0	0.000	0.098
late	corm_no_safety	18.8	0.150	27.5	0.583	0.092
late	corm_no_uncertainty	28.7	0.130	15.0	0.920	0.073
late	corm_repair_memory_v5	20.0	0.133	16.2	0.593	0.089
late	global_average_repair	15.0	0.159	28.7	0.525	0.124
late	knn_repair_memory_v4	26.2	0.137	25.0	0.604	0.092
late	last_repair_memory	7.5	0.176	30.0	0.056	0.146
late	nominal_mpc	7.5	0.177	30.0	0.000	0.149
late	online_ridge_residual	23.8	0.154	28.7	0.920	0.084
late	oracle_hidden_deployment	61.3	0.000	0.0	0.000	0.118
late	random_candidate	5.0	0.346	52.5	0.000	0.133
late	robust_worst_case_mpc	12.5	0.152	23.8	0.000	0.120
middle	corm_no_safety	13.6	0.150	33.0	0.513	0.087
middle	corm_no_uncertainty	14.8	0.150	25.0	0.920	0.076
middle	corm_repair_memory_v5	13.6	0.140	26.1	0.523	0.087
middle	global_average_repair	12.5	0.171	36.4	0.520	0.114
middle	knn_repair_memory_v4	12.5	0.155	31.8	0.516	0.104
middle	last_repair_memory	10.2	0.179	34.1	0.056	0.134
middle	nominal_mpc	9.1	0.185	35.2	0.000	0.136
middle	online_ridge_residual	18.2	0.146	30.7	0.920	0.086
middle	oracle_hidden_deployment	53.4	0.000	0.0	0.000	0.101
middle	random_candidate	1.1	0.325	53.4	0.000	0.155
middle	robust_worst_case_mpc	12.5	0.155	27.3	0.000	0.110

Table 31: Early/middle/late learning diagnostics for obstacle_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	4.5	0.164	37.5	0.223	0.100
early	corm_no_uncertainty	5.7	0.161	28.4	0.535	0.094
early	corm_repair_memory_v5	4.5	0.164	37.5	0.223	0.100
early	global_average_repair	4.5	0.172	39.8	0.197	0.108
early	knn_repair_memory_v4	5.7	0.174	39.8	0.144	0.108
early	last_repair_memory	4.5	0.174	39.8	0.051	0.114
early	nominal_mpc	5.7	0.180	40.9	0.000	0.111
early	online_ridge_residual	4.5	0.163	34.1	0.535	0.098
early	oracle_hidden_deployment	34.1	0.000	0.0	0.000	0.127
early	random_candidate	1.1	0.325	58.0	0.000	0.157
early	robust_worst_case_mpc	9.1	0.147	30.7	0.000	0.103
late	corm_no_safety	1.2	0.164	36.2	0.583	0.082
late	corm_no_uncertainty	5.0	0.144	13.8	0.920	0.056
late	corm_repair_memory_v5	1.2	0.148	23.8	0.599	0.071
late	global_average_repair	2.5	0.188	42.5	0.522	0.114
late	knn_repair_memory_v4	1.2	0.183	41.2	0.597	0.089
late	last_repair_memory	2.5	0.192	41.2	0.056	0.122
late	nominal_mpc	2.5	0.192	41.2	0.000	0.124
late	online_ridge_residual	5.0	0.161	31.2	0.920	0.077
late	oracle_hidden_deployment	32.5	0.000	0.0	0.000	0.140
late	random_candidate	0.0	0.312	52.5	0.000	0.119
late	robust_worst_case_mpc	2.5	0.174	36.2	0.000	0.105
middle	corm_no_safety	6.8	0.164	30.7	0.500	0.101
middle	corm_no_uncertainty	5.7	0.135	10.2	0.920	0.064
middle	corm_repair_memory_v5	6.8	0.158	27.3	0.506	0.093
middle	global_average_repair	5.7	0.175	36.4	0.503	0.116
middle	knn_repair_memory_v4	5.7	0.170	31.8	0.503	0.112
middle	last_repair_memory	5.7	0.172	33.0	0.056	0.122
middle	nominal_mpc	6.8	0.165	31.8	0.000	0.124
middle	online_ridge_residual	5.7	0.162	25.0	0.920	0.097
middle	oracle_hidden_deployment	38.6	0.000	0.0	0.000	0.129
middle	random_candidate	1.1	0.307	45.5	0.000	0.140
middle	robust_worst_case_mpc	3.4	0.154	28.4	0.000	0.101

Table 32: Early/middle/late learning diagnostics for nonstationary_deployment_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	8.0	0.217	42.0	0.202	0.166
early	corm_no_uncertainty	5.7	0.212	37.5	0.535	0.146
early	corm_repair_memory_v5	8.0	0.215	40.9	0.202	0.166
early	global_average_repair	8.0	0.217	42.0	0.169	0.172
early	knn_repair_memory_v4	10.2	0.221	43.2	0.131	0.175
early	last_repair_memory	6.8	0.226	43.2	0.051	0.177
early	nominal_mpc	9.1	0.222	43.2	0.000	0.174
early	online_ridge_residual	4.5	0.216	40.9	0.535	0.152
early	oracle_hidden_deployment	48.9	0.000	1.1	0.000	0.128
early	random_candidate	1.1	0.310	40.9	0.000	0.182
early	robust_worst_case_mpc	8.0	0.192	36.4	0.000	0.146
late	corm_no_safety	17.5	0.170	33.8	0.490	0.127
late	corm_no_uncertainty	11.2	0.155	18.8	0.920	0.109
late	corm_repair_memory_v5	17.5	0.171	32.5	0.480	0.133
late	global_average_repair	18.8	0.182	36.2	0.456	0.135
late	knn_repair_memory_v4	20.0	0.164	32.5	0.540	0.122
late	last_repair_memory	13.8	0.178	31.2	0.056	0.142
late	nominal_mpc	13.8	0.179	31.2	0.000	0.142
late	online_ridge_residual	17.5	0.191	40.0	0.920	0.123
late	oracle_hidden_deployment	45.0	0.000	0.0	0.000	0.133
late	random_candidate	1.2	0.318	52.5	0.000	0.140
late	robust_worst_case_mpc	15.0	0.149	26.2	0.000	0.104
middle	corm_no_safety	4.5	0.212	44.3	0.413	0.156
middle	corm_no_uncertainty	1.1	0.205	31.8	0.920	0.128
middle	corm_repair_memory_v5	4.5	0.223	45.5	0.411	0.153
middle	global_average_repair	5.7	0.222	47.7	0.435	0.152
middle	knn_repair_memory_v4	4.5	0.232	51.1	0.457	0.152
middle	last_repair_memory	4.5	0.218	46.6	0.056	0.162
middle	nominal_mpc	4.5	0.217	45.5	0.000	0.159
middle	online_ridge_residual	5.7	0.183	36.4	0.920	0.142
middle	oracle_hidden_deployment	39.8	0.000	0.0	0.000	0.129
middle	random_candidate	0.0	0.314	39.8	0.000	0.160
middle	robust_worst_case_mpc	6.8	0.178	34.1	0.000	0.125

Table 33: Early/middle/late learning diagnostics for combined_shift.

Phase	Method	Success	Regret	Violation	Trust	Pred. error
early	corm_no_safety	6.8	0.223	48.9	0.215	0.161
early	corm_no_uncertainty	5.7	0.202	37.5	0.535	0.144
early	corm_repair_memory_v5	6.8	0.223	48.9	0.215	0.160
early	global_average_repair	4.5	0.236	51.1	0.166	0.181
early	knn_repair_memory_v4	5.7	0.231	50.0	0.132	0.178
early	last_repair_memory	4.5	0.240	50.0	0.051	0.191
early	nominal_mpc	3.4	0.236	48.9	0.000	0.183
early	online_ridge_residual	5.7	0.209	45.5	0.535	0.150
early	oracle_hidden_deployment	39.8	0.000	0.0	0.000	0.117
early	random_candidate	2.3	0.312	58.0	0.000	0.181
early	robust_worst_case_mpc	3.4	0.199	40.9	0.000	0.151
late	corm_no_safety	10.0	0.191	42.5	0.510	0.125
late	corm_no_uncertainty	5.0	0.171	21.2	0.920	0.090
late	corm_repair_memory_v5	7.5	0.193	32.5	0.512	0.125
late	global_average_repair	6.2	0.223	45.0	0.450	0.164
late	knn_repair_memory_v4	6.2	0.204	42.5	0.537	0.141
late	last_repair_memory	5.0	0.230	47.5	0.056	0.170
late	nominal_mpc	5.0	0.233	47.5	0.000	0.173
late	online_ridge_residual	8.8	0.184	37.5	0.920	0.100
late	oracle_hidden_deployment	48.8	0.000	2.5	0.000	0.159
late	random_candidate	2.5	0.274	41.2	0.000	0.172
late	robust_worst_case_mpc	7.5	0.194	35.0	0.000	0.151
middle	corm_no_safety	12.5	0.207	46.6	0.478	0.125
middle	corm_no_uncertainty	9.1	0.192	27.3	0.920	0.097
middle	corm_repair_memory_v5	9.1	0.203	34.1	0.464	0.119
middle	global_average_repair	5.7	0.223	43.2	0.439	0.164
middle	knn_repair_memory_v4	8.0	0.204	39.8	0.466	0.143
middle	last_repair_memory	4.5	0.236	45.5	0.056	0.176
middle	nominal_mpc	5.7	0.246	47.7	0.000	0.182
middle	online_ridge_residual	12.5	0.209	45.5	0.920	0.118
middle	oracle_hidden_deployment	46.6	0.000	2.3	0.000	0.129
middle	random_candidate	0.0	0.263	35.2	0.000	0.157
middle	robust_worst_case_mpc	6.8	0.209	36.4	0.000	0.153

effect, so CORM must show more than a mean correction. The relevant question is whether context features and online ridge structure improve action ranking over global repair without increasing obstacle violations. If the generated tables show only a small margin over global repair, the correct interpretation is that the deployment contains reusable residual structure but not enough structure to justify the full mechanism.

High-friction split. High friction reverses the direction of the low-friction bias. It is a useful check against asymmetric tuning. A repair memory tuned to only add distance can look good under low friction and fail under high friction. CORM's residual regression can represent either sign, but the finite stream length limits how quickly it can identify the sign. This split should be read together with the early/middle/late learning tables: a method that begins poorly but improves late is different from a method that remains flat.

Heavy-object split. Heavy objects combine mass and friction effects. The same push distance can produce a smaller displacement and different contact settling behavior. This split is where online system identification should, in principle, help. The failure mode is that selected-action residuals are biased by the policy's own choices: if the method avoids certain offsets or angles early, it receives no residual evidence about them. This is an inherent partial-feedback problem and a reason to avoid overstating the result.

Light-object split. Light objects often make actions more aggressive and can increase safety risk near obstacles. A low-regret method that becomes unsafe here is not acceptable. The violation-rate table is therefore as important as the success table. The no-uncertainty ablation is especially relevant: if full trust increases success while safety degrades, the paper should treat that as an overconfidence failure, not as a better method.

Actuation-bias split. Actuation bias is the most natural setting for residual repair because an angle or lateral bias is persistent across tasks. A linear residual model should be able to identify part of it from repeated observations. If CORM fails to separate from online ridge residual here, the calibration layer is not adding enough beyond ordinary online regression. That is one reason the frozen gate includes online ridge as a strong baseline.

Obstacle-shift split. Obstacle shift exposes a limitation of residual displacement memories. The hidden error is not only in puck displacement; it is also in the safety geometry. A residual that corrects final position may still choose an action whose true obstacle clearance is worse than nominal. This split motivates future work on explicit scene-geometry repair rather than only outcome repair. A strong result on final distance but weak violation behavior should not be treated as a submission-ready win.

Nonstationary deployment split. The nonstationary split is the harshest test of persistent memory. A memory that helps before the change point can become stale afterward. CORM uses a shock variable to reduce trust after unexpectedly large residual prediction errors, but this is only a simple detector. The frozen decision audit shows why this split matters: if safety worsens relative to robust MPC after a hidden change, the method cannot claim reliable deployment memory. The correct conclusion is not that memory is useless, but that stale-memory detection is a first-class part of the problem.

Combined shift. The combined split is a stress mixture, not a clean mechanism probe. It tests whether all components interact safely: residual estimation, trust calibration, candidate ranking, and obstacle handling. A method that wins one clean shift but fails combined shift is not ready for submission as a general deployment-memory system. The combined split is also where the oracle gap is most informative. A large oracle gap means the candidate library contains better choices, but the policy cannot identify them from available evidence.

APPENDIX E: ARTIFACT SCHEMA AND REPRODUCIBILITY DETAILS

The artifact is designed so that a reviewer can audit the claim without trusting the prose. The main raw CSV, 'repair_memory_raw.csv', is the authoritative evidence table. Each row corresponds to one method decision on one paired task. The key columns are:

Identification columns. ‘seed’, ‘episode’, ‘phase’, ‘deployment_phase’, ‘split’, and ‘method’ identify the paired case. The same seed, episode, and split can be used to compare methods directly because all methods receive the same candidate set and hidden deployment.

Hidden deployment columns. ‘true_mass’, ‘true_friction’, ‘distance_gain’, ‘angle_bias’, and ‘lateral_bias’ record the deployment parameters used by the simulator. These are logged for audit, not exposed to deployable methods. Only the oracle uses hidden-deployment rollouts for selection.

Task and action columns. ‘initial_distance’, ‘candidate_count’, and ‘chosen_action’ describe the task scale and selected primitive. The frozen candidate count is 63. Candidate count is logged to catch accidental action-library changes.

Outcome columns. ‘success’, ‘final_distance’, ‘violation’, ‘energy’, ‘oracle_energy’, and ‘energy_regret’ are the primary evaluation quantities. Regret is always computed against the same hidden-deployment oracle and is never normalized post hoc.

Memory diagnostic columns. ‘decision_trust’, ‘decision_uncertainty’, ‘predicted_violation_risk’, ‘shield_active’, ‘stale_score’, and ‘repair_prediction_error’ expose the internal state of repair methods. Non-memory baselines have zero trust and shield values. These diagnostics are important because a memory method can appear to improve success while becoming overconfident or unsafe.

Ablation schema. ‘repair_memory_ablation_raw.csv’ uses the same schema but only on combined and nonstationary splits. Keeping the same schema prevents a common ablation mistake: reporting a different metric set for weakened methods than for the proposed method.

Generated summaries. ‘repair_memory_metrics.csv’ aggregates by split and method. ‘repair_memory_seed_metrics.csv’ keeps seed-level summaries. ‘repair_memory_pairwise.csv’ contains CORM deltas against every main baseline for each split. ‘repair_memory_learning_curve.csv’ aggregates early, middle, and late stream phases. ‘repair_memory_calibration.csv’ records trust and prediction-error diagnostics.

Validation. The validation script checks row counts, PDF placement, page count, and Desktop absence. It is intentionally mechanical. It cannot decide whether the idea is good, but it can catch missing evidence, accidental short PDFs, and artifact-placement violations.

APPENDIX F: WHAT WOULD BE REQUIRED TO REVIVE THE PAPER

The frozen decision is **KILL_ARCHIVE**. A future revival should not simply lower the gate. It should address the mechanisms exposed by the kill/archive result.

First, stale-memory detection must be explicit. The shock variable is a useful diagnostic, but it is not a principled change-point model. A revived method should compare persistent-memory, reset, and detected-change variants under controlled hidden change points. It should report detection delay, false reset rate, and safety during the transition period.

Second, repair should include scene geometry, not only outcome residuals. Obstacle-shift failures show that final-position correction is insufficient. A stronger method would maintain a belief over obstacle displacement or clearance residuals and use it inside a safety-aware planner. That belief should be evaluated with calibration metrics, not only downstream success.

Third, exploration bias must be handled. CORM observes residuals only for executed actions. This creates a partial-feedback loop: the memory improves where the policy already acts and remains uncertain elsewhere. A revival should test safe information-gathering pushes or conservative Thompson-style action selection. Any exploration mechanism must pay its own cost in the reported energy or task budget.

Fourth, the robust baseline should get stronger, not weaker. The robust MPC baseline here uses a small branch set. A future paper should include richer branch uncertainty, CVaR-style scoring, and possibly sampling-based model predictive path integral control (Williams et al., 2017). If repair memory cannot beat stronger robust planners, the practical contribution is limited.

Fifth, the evidence must leave the custom simulator. The current result is useful because it is controlled and reproducible, but it is not enough for ICLR-main readiness. A revived paper should evaluate on a public manipulation benchmark such as CALVIN or FurnitureBench (Mees et al.,

2022; Heo et al., 2025), or on hardware with repeated deployment shifts that can be independently inspected.

Sixth, the claim should remain narrow. The safe claim is not that robots need memory in general; it is that calibrated, uncertainty-aware residual memory can help under recurring physical deployment shifts when stale-memory and safety failure modes are controlled. If the evidence does not support that claim, the paper should remain archived.

REFERENCES

Agnar Aamodt and Enric Plaza. Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI Communications*, 7(1):39–59, 1994.

Brian D. O. Anderson and John B. Moore. *Optimal Filtering*. Prentice Hall, 1979.

Alberto Bemporad and Manfred Morari. Robust model predictive control: A survey. *Robustness in Identification and Control*, pp. 207–226, 1999.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.

Marc Peter Deisenroth and Carl Edward Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning*, 2011.

Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast reinforcement learning via slow reinforcement learning. In *arXiv preprint arXiv:1611.02779*, 2016.

Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*, 2017.

Minho Heo, Youngwoon Lee, Doohyun Lee, and Joseph J. Lim. Furniturebench: Reproducible real-world benchmark for long-horizon complex manipulation. *The International Journal of Robotics Research*, 2025.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, 2019.

Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, 2 edition, 1999.

David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

David Q. Mayne, Maria M. Seron, and Sasa V. Rakovic. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005.

Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. In *IEEE Robotics and Automation Letters*, 2022.

Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE International Conference on Robotics and Automation*, 2018.

Adam Santoro, David Raposo, David G. T. Barrett, Mateusz Malinowski, Razvan Pascanu, Peter Battaglia, and Timothy Lillicrap. Relational recurrent neural networks. In *International Conference on Machine Learning*, 2018.

Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation*, 2017.

Table 34: CORM v5 trust calibration by split and stream phase.

Split	Phase	Method	Trust	Uncertainty	Pred. error	Shield
actuation_bias_shift	early	CORM	0.222	0.179	0.100	1.1
actuation_bias_shift	late	CORM	0.593	0.113	0.089	2.5
actuation_bias_shift	middle	CORM	0.523	0.120	0.087	1.1
combined_shift	early	CORM	0.215	0.192	0.160	1.1
combined_shift	late	CORM	0.512	0.129	0.125	5.0
combined_shift	middle	CORM	0.464	0.136	0.119	2.3
heavy_object_shift	early	CORM	0.234	0.178	0.090	0.0
heavy_object_shift	late	CORM	0.631	0.099	0.062	3.8
heavy_object_shift	middle	CORM	0.557	0.108	0.062	1.1
high_friction_shift	early	CORM	0.213	0.199	0.131	0.0
high_friction_shift	late	CORM	0.588	0.128	0.085	0.0
high_friction_shift	middle	CORM	0.493	0.142	0.095	1.1
light_object_shift	early	CORM	0.226	0.186	0.113	0.0
light_object_shift	late	CORM	0.587	0.115	0.079	1.2
light_object_shift	middle	CORM	0.533	0.118	0.089	1.1
low_friction_shift	early	CORM	0.224	0.189	0.090	0.0
low_friction_shift	late	CORM	0.622	0.111	0.076	3.8
low_friction_shift	middle	CORM	0.533	0.123	0.074	2.3
nominal	early	CORM	0.237	0.175	0.049	0.0
nominal	late	CORM	0.673	0.086	0.036	0.0
nominal	middle	CORM	0.569	0.102	0.033	1.1
nonstationary_deployment_shift	early	CORM	0.202	0.204	0.166	0.0
nonstationary_deployment_shift	late	CORM	0.480	0.166	0.133	2.5
nonstationary_deployment_shift	middle	CORM	0.411	0.160	0.153	1.1
obstacle_shift	early	CORM	0.223	0.182	0.100	0.0
obstacle_shift	late	CORM	0.599	0.112	0.071	3.8
obstacle_shift	middle	CORM	0.506	0.123	0.093	2.3